

UNIVERSIDAD
DE GRANADA

Práctica 2

Problema del portfolio

Pablo Linari Pérez

DNI: 75571079W
Grupo de prácticas: 3 (Martes 15:30-17:30)
Curso: 25/26
Correo: pablolinari@correo.ugr.es

Índice de contenidos

1 Descripción del problema 3
2 Algoritmos empleados y estructura del problema 4
2.1 Estructura del problema 4
2.2 Representación de las soluciones 4
2.3 Funciones comunes usadas por los algoritmos 5
2.4 Operadores 7
2.4.1 Operadores de cruce 7
2.4.2 Operador de selección 9
2.4.3 Operador de mutación 9
3 Estructura de los algoritmos 11
3.1 Local Search 11
3.2 Algoritmo Genético Generacional 12
3.3 Algoritmo Genético Estacionario 13
3.4 Algoritmo Memético 14
3.4.1 AM-Random 14
3.4.2 AM-All 14
3.4.3 AM-Best 14
4 Estructura del código 15
4.1 Organización de archivos 15
4.2 Compilación 16
4.3 Manual de usuario: Ejecución 16
5 Experimentos y análisis de los resultados 17
5.1 Casos empleados y valores de los parámetros. 17
5.2 Resultados obtenidos 18
5.3 Boxplots de los resultados 19
6 Análisis de los resultados 20
6.1 Comportamiento de los Algoritmos Genéticos Generacionales 20
6.2 Comportamiento de los Algoritmos Genéticos Estacionarios 21
6.3 Comportamiento de los Algoritmos Meméticos 21
6.4 Mejor operador para los genéticos 21
6.5 Búsqueda Local vs Algoritmos Genéticos 21
6.6 AGE-BLα vs AGG-BLα 21
6.7 Comparación entre meméticos 22
6.8 Meméticos vs mejor AG 22
6.9 Comparación gráfica fitness 2014-2024 (50 ejecuciones) 22
6.10 Comparación global y conclusiones 23

1 Descripción del problema

El problema que se ha escogido es el problema del portfolio, en el cual se nos da una lista de empresas de las cuales sabemos su retorno obtenido diario desde principios de 2015 hasta finales de 2025. Con estos datos necesitamos elaborar una solución indicando qué porcentaje de nuestra cartera deberíamos haber invertido en cada empresa para maximizar el beneficio de las inversiones.

En este contexto, el problema no solo busca el máximo retorno, sino que también suele considerar la correlación entre activos, teniendo esto en cuenta en las ecuaciones que se usan para maximizar el beneficio.

De esta manera podemos ajustar el programa ajustando el parámetro λ de la ecuación a maximizar para así tener mayor o menor aversión al riesgo.

2 Algoritmos empleados y estructura del problema

2.1 Estructura del problema

Para modelar el problema del portfolio se ha usado la clase Pportfolio la cual es hija de la clase Problem que viene ya predefinida con varios métodos virtuales, los cuales se implementan en la clase hija. Para representar el problema usando estas clases se tienen en cuenta los siguientes atributos:

- **days** : número entero de días de los que se tienen datos.
- **ncompany** : número entero de empresas de las que se tienen datos.
- **low** : mínimo porcentaje de inversión en una empresa (double).
- **high** : máximo porcentaje de inversión en una empresa (double).
- **lambda** : parámetro entero usado en el fitness.
- **data** : matriz de la librería Eigen con valores double que contienen los beneficios diarios de cada empresa, en las columnas se representan las empresas y en las filas los días.
- **covarianceMat** : matriz de la librería Eigen con valores double que contiene la covarianza de las empresas.
- **logSums** : vector de la librería Eigen con valores double que contiene la suma por columnas después de aplicar **log(valor+1)** a cada posición de la matriz.
- **ERROR** : Error permitido en la suma total del vector solución (double).

Podemos observar que se ha optado por usar la librería Eigen de C++ para poder manejar las operaciones de matrices y vectores como la suma total, la media total, la media por columnas o por filas, multiplicación escalar a escalar, etc. Esto hace que el código sea más compacto y fácil de seguir a la vez que más preciso y cómodo de utilizar.

También se han guardado los datos como **covarianceMat** y **logSums** que son constantes durante la ejecución del programa y solo cambian cuando cambia el archivo de entrada. Al ser valores calculados con operaciones matriciales son costosos de calcular y eso hace que el programa sea más rápido.

2.2 Representación de las soluciones

Para este problema las soluciones se representan por vectores de la biblioteca Eigen con tipo de dato double donde cada posición del vector representa la i-ésima empresa del fichero de datos de entrada, todos estos vectores deben cumplir las siguientes restricciones:

- El tamaño de la solución debe ser el mismo que el número de empresas en los datos de entrada.
- La suma de todos sus valores debe ser igual a 1.
- Cada posición del vector tiene que tener un valor entre **low** y **high**
- Puede haber posiciones del vector con valor 0 (representando que en esa empresa no se invierte).

La mejor solución será aquella que consiga maximizar la función objetivo.

2.3 Funciones comunes usadas por los algoritmos

En primer lugar la función que permite cargar los datos en nuestra clase Pportfolio es una función implementada en un archivo auxiliar ya que no tiene por qué pertenecer necesariamente al problema que se plantea. Esta función lee datos de un fichero en formato CSV. Tras abrir el archivo la función descarta la primera línea con los nombres de las empresas y comienza a leer las siguientes líneas, una vez escogida una línea, se salta el string que contiene la fecha y comienza a insertar datos al vector usando como separador la coma. El código comprueba los valores de la fecha leída, si el parámetro only2025 es True, solo se leen datos de 2025, de lo contrario se leen todos los datos hasta el último día del 2024. Una vez leídos todos los datos, la matriz obtenida se convierte a una matriz Eigen para poder manejarla con más facilidad.

```
funcion LoadCSV(archivo, only2025):
  matrix data
  leer(primera_linea)
  abrir archivo
  while(leer(linea_archivo)):
    if es fecha el primer string :
      continue
    if(es fecha el primer string and not only2025):
      solo leer valores de 2014-2024
    if(es fecha el primer string and only2025):
      solo leer valores de 2025
    vector valores

     while(leer(dato)):
       if(dato no vacío):
         valores.pushback(dato)
       else:
         valores.pushback(0.0)

    data.pushback(valores)

  cerrar archivo

   if(data no vacío):
     matriz_eigen = data
  return matriz_eigen
```

Las principales funciones que se usan de la clase **Pportfolio** para implementar los algoritmos son las siguientes:

La función **createSolution** se usa en dos algoritmos, en la Búsqueda Aleatoria para generar soluciones aleatorias y escoger finalmente la que mejor fitness tenga y en la Búsqueda Local para generar la solución desde la cual se comienza a buscar la mejor solución posible.

```
funcion createSolution():
   vector solución;

   for i<solución.size :
```

```
      solución(i) = Aleatorio entre high y low


    numeroceros = Aleatorio entre 1 y solución.size-1
    vectorindices = {0,1,2, ......,solución.size-1}
   desordenar(vectorindices)
    ## Seleccionamos valores aleatorios que ponemos a 0
    for i<numeroceros:
      solución(vectorindices[i])=0

    solución = solución/solución.sum
    if la solución no es válida
      fix(solución)

    return solución;
```

```
# Pseudocódigo del arreglo de solución
 funcion fix(solución):
    solución = solución/solución.sum

 for i in ncompany:
   if solución(i) < low:
     solución(i) = 0
   else if solución(i) > high:
     solución(i) = this->high


 double diff = 1.0 - solución.sum
 while diff >ERROR and no supera máximo de iteraciones:
  ajustado = false

   while i < ncompany and not ajustado:
     if solución(i) en rango [low, high):
       espacio = high - solución(i)
      if hay espacio :
        sumar diferencia y ajustado=true
       else :
         solución(i) = high
        diff =diff- espacio

   if no se ha ajustado y diferencia menor que low:
     for i in ncompany and not ajustado:
       if solución(i) == 0:
         solución(i) = mínimo entre diff y high
        ajustado = true

   if no se ha ajustado y la diferencia es menor que low:
     while i < ncompany and not ajustado:
       if (solución(j) > low and solución(j) + diff < high:
         solución(j) += diff
        ajustado = true

   diff = 1.0 - solución.sum
```

La solución fix se aplica cada vez que se genera una solución aleatoria para arreglarla, lo que se hace es tras normalizar la solución modificar los valores fuera de rango para que cumplan las condiciones de ser solución. Una vez hecho esto, si hace falta añadir más valor en alguna posición se añade, primero se comprueba si se puede repartir dentro de pesos que ya estén activos, de no ser así si la diferencia es mayor que el límite inferior se activa un cero de la solución. En caso de que la diferencia sea menor que el mínimo permitido se intenta distribuir la diferencia entre valores que ya estén activos y tengan un margen para ser de nuevo ajustados. Este proceso se repite hasta que la diferencia es menor que el error permitido o se sobrepasen las evaluaciones. En el código las evaluaciones permitidas son altas para que no haya problema por la precisión de las operaciones.

```
# Pseudocódigo del cálculo del beneficio
 funcion profit(solution):
   return (solution * logSums).sum #multiplicación componente a componente
```

```
# Pseudocódigo del cálculo del riesgo
 funcion risk(solution):
   return (solution.transpose * covarianceMat * solution);
#devuelve un número resultado de las dimensiones 1xn * nxn * nx1
```

```
# Pseudocódigo del cálculo del fitness
 funcion fitness(solution):
   return (profit - lambda*risk);
#devuelve un número resultado de las dimensiones 1xn * nxn * nx1
```

El principal objetivo de los algoritmos es maximizar la función fitness, la cual trata de maximizar el beneficio y minimizar el riesgo en una sola fórmula. El parámetro lambda se usará a modo de ajuste, cuando más alto sea su valor el algoritmo valorará mejor soluciones con poco beneficio pero que sean seguras, es decir, que tengan poco riesgo, mientras que cuanto más cercano a 0 sea lambda, el algoritmo valorará mejor las soluciones con máximo beneficio posible, ignorando el riesgo.

2.4 Operadores

Para agrupar los operadores usados en los diferentes algoritmos he utilizado una clase **Operator** la cual tiene como métodos públicos los operadores usados en los algoritmos genéticos y meméticos.

2.4.1 Operadores de cruce

En este problema se han utilizado dos operadores de cruce, **Aritmético** y **BLX-α**

En ambos operadores, si los padres seleccionados son iguales, los hijos serán iguales a los padres.

El operador aritmético genera un aleatorio entre 0 y 1, para cada posición **i** de los padres, con el aleatorio escogido hace una combinación lineal distinta para cada hijo entre los genes de los padres. Si los genes de los padres son iguales en una determinada posición,

no se genera la transformación, de esta manera conseguimos que si todos los genes de los padres son iguales, los hijos sean iguales.

Al ser este cruce una combinación lineal, los genes de los hijos resultantes, nunca podrán tener un valor menor que el menor de los padres ni mayor que el mayor de los genes de sus padres. Esto hace que este operador tienda a converger hacia el centro de los padres, con un comportamiento explotativo.

```
# Pseudocódigo del operador aritmético
 funcion arithmetic(padre1 , padre2):
   hijo1(padre1.size) ## genero los hijos con el mismo tamaño que los padres
   hijo2(padre1.size) ## en el problema que tratamos todos los cromosomas
miden lo mismo

     for i in range(0,tamaniopadres):

     if (padre1[i] == padre2[i]):
     # asigno el mismo gen a los hijos
     # de esta forma si los padres son idénticos
     # sus hijos también los son
     hijo1[i]=padre1[i]
     hijo2[i]=padre1[i]
     else:
     sigma = double aleatorio entre 0 y 1
     hijo1[i] =sigma*padre1[i]+(1-sigma)*padre2[i]
     hijo2[i] =sigma*padre2[i]+(1-sigma)*padre1[i]
    return pareja de hijos (hijo1, hijo2)
```

El operador BLX-alpha genera un intervalo entre el mínimo y el máximo de los genes seleccionados de los padres, a este intervalo se le añade un parámetro **alpha*gap**, el cual permite que el intervalo se agrande según el **alpha** escogido y la distancia entre el mínimo y el máximo. Esto permite que el valor aleatorio que se genere en ese intervalo para cada hijo, pueda ser más grande o más pequeño que los valores que el padre tenía por defecto, por lo que permite explorar más soluciones y mantiene más diversidad ya que se generan genes que están fuera de los rangos de los padres.

```
# Pseudocódigo del operador BLX-alpha
 funcion arithmetic(padre1 , padre2):
   hijo1(padre1.size) ## genero los hijos con el mismo tamaño que los padres
   hijo2(padre1.size) ## en el problema que tratamos todos los cromosomas
miden lo mismo

    for i in range(0,tamaniopadres):

    if(padre1[i] == padre2[i]):
     # asigno el mismo gen a los hijos
     # de esta forma si los padres son idénticos
     # sus hijos también los son
     hijo1[i]=padre1[i]
     hijo2[i]=padre1[i]
    else:
      cmax = double máximo entre padre1[i],padre2[i]
      cmin = double mínimo entre padre1[i],padre2[i]
```

```
    gap = cmax-cmin
    hijo1[i] = aleatorio entre (cmin-gap*alpha,cmax+gap*alpha)
    hijo2[i] = aleatorio entre (cmin-gap*alpha,cmax+gap*alpha)
  return pareja de hijos (hijo1, hijo2)
```

Ambos operadores devuelven parejas de hijos que pueden no ser válidos, por lo que se tendrán que arreglar ambos cuando estos se usen en los correspondientes algoritmos.

2.4.2 Operador de selección

Para la selección de los padres se usa un torneo entre tres cromosomas elegidos al azar de la población actual. Estos tres cromosomas compiten por ver quién tiene mejor fitness y el que gane se convierte en padre. La función recibe como parámetros el número de seleccionados por torneo (k), el tamaño de la población, el número de padres que se deben de seleccionar y el vector de fitness de la población, para economizar las evaluaciones. Como resultado la función devuelve un vector de índices de los cromosomas escogidos para ser padres.

```
# Pseudocódigo del operador tournament
  funcion tournament(tournamentsize, populationsize, selectionsize,
fitnesspoblation):
  vector idxparents # vector con los índices de los padres
  # repito el torneo tantas veces como necesite
      for i in range(0, selectionsize):
      idxbest = 0

      for j in range(0, tournamentsize):
        idx = aleatorio entre 0 y populationsize-1
      #selecciono el mejor de entre los participantes del torneo
        if fitnesspoblation[idx] > fitnessTournament:
          fitnessTournament = fitnesspoblation[idx]
          idxBest = idx
      #añado el mejor del torneo a la lista de padres
      idxparents.pushback(idxBest)
  return idxparents
```

2.4.3 Operador de mutación

A la hora de hacer la mutación en los algoritmos usaremos el siguiente método:

```
# Pseudocódigo del operador de mutación
funcion mutation(mutSol, problem, mutationRate, maxAttempts):
  problembounds = problem.getSolutionDomainRange()
  solutionsize = problem.getSolutionSize()

  validDonors = []
  for i in range(0, solutionsize):
    amount = mutSol(i) * mutationRate
    if amount <= 0.0:
      continuar
    firstAfter = mutSol(i) - amount
    firstValid = firstAfter == 0.0 o
```

```
                (firstAfter >= problembounds.first y
                 firstAfter <= problembounds.second)
    if firstValid:
      validDonors.pushback(i)

  if validDonors está vacío:
    return false

  for attempt in range(0, maxAttempts):
    pickDonor = aleatorio entre 0 y validDonors.size - 1
    candidateFirst = validDonors[pickDonor]
    amount = mutSol(candidateFirst) * mutationRate

    validReceivers = []
    for i in range(0, solutionsize):
      if i == candidateFirst:
        continuar
      secondAfter = mutSol(i) + amount
      secondValid = secondAfter == 0.0 o
                    (secondAfter >= problembounds.first y
                     secondAfter <= problembounds.second)
      if secondValid:
        validReceivers.pushback(i)

    if validReceivers está vacío:
      continuar

    idx = aleatorio entre 0 y validReceivers.size - 1
    candidateSecond = validReceivers[idx]

    mutSol(candidateFirst) -= amount
    mutSol(candidateSecond) += amount
    return true

  return false
```

La función de mutación recibe como parámetros la solución a mutar (por referencia), un objeto problema, el ratio de mutación y el número máximo de intentos que tenemos para elegir tanto el primer como el segundo gen.

Como se puede observar, el proceso de mutación se divide en dos partes:

- En el primer bucle se buscan todos los genes a los cuales se les puede restar el porcentaje indicado en la variable **mutationrate** y que siguen cumpliendo las restricciones del problema.Si no se encuentran posibles donantes se acaba la ejecución.
- En el segundo bucle, se selecciona un gen donante del vector calculado previamente y se buscan todos los posibles genes que puedan recibir la cantidad que se le va a restar a este gen.Si no se encuentra ningún gen que pueda recibir dicha cantidad , se vuelve a escoger un donante aleatorio y se repite el proceso. Este proceso se repite todas las veces que permita el limite de intentos marcado por **maxattempts** . En la práctica , tras varias pruebas el límite se ha puesto lo suficientemente alto para que se pueda encontrar el gen receptor.

3 Estructura de los algoritmos

3.1 Local Search

El algoritmo de búsqueda local (el primero el mejor) parte de una solución generada aleatoriamente y genera un vecindario alrededor de esta solución, una vez generado el vecindario, se comienza a recorrer cada vecino y se evalúa su fitness. Si se encuentra un fitness mejor que el de la solución actual, se escoge ese vecino como mejor solución y se genera un nuevo vecindario alrededor de él. El proceso termina cuando se llega al máximo de evaluaciones permitidas o cuando no se ha encontrado ningún vecino del vecindario actual generado que mejore la solución.

```
# Pseudocódigo del algoritmo de búsqueda local
función LocalSearch(max_evals):
    double ratio
     solucioninicial = solucionAleatoria
     bestFitness = fitness(solucionAleatoria)

    vecinos = Generar vector de todas las parejas (i,j) posibles con i,j en
[0,ncompany-1]
    # excepto en el caso i=j;

    while evals < maxevals and haymejora:

      mezclar vecinos
      for v in vecinos:
        cantidadmover = ratio*solucioninicial[v.primero]

        solucioninicial[v.primero]-=cantidadmover
        solucioninicial[v.segundo]+=cantidadmover

         if es válida solucioninicial:
          evals = evals+1

           if fitness(solucioninicial) > mejorfitness:
             haymejora = true
             mejorfitness = fitness de la solución inicial modificada
             #si hay una mejora salimos del bucle for
             #ya que es una búsqueda del primero el mejor
            salir del bucle for
         # si no cambiamos de solución inicial, hay que
         # devolverla a su estado original
         devolver solucioninicial a su estado original
         if evals==maxevals:
          salir del bucle for
          #para no pasarnos de evaluaciones
     return solucioninicial, mejorfitness, evals
```

```
   # Generación del vecindario
  for i,j in ncompany:
    vectorparejas.insertar(pareja(i,j))
```

La implementación del algoritmo de Búsqueda local se divide en dos partes, primero se crea el vector vecinos, en este caso se usa un vector de parejas, donde cada vecino es la pareja (i,j) la cual nos indicará los índices que debemos modificar en el vector solución. Una vez creado el vector, lo mezclamos aleatoriamente, y este será el que nos indique qué vecinos debemos de probar. Si no fuese una búsqueda del primero el mejor y se escogiese el mejor vecino no haría falta mezclar las parejas, ya que el orden no importaría porque siempre tendríamos que recorrer todo el vecindario.

Una vez disponemos del vecindario, debemos generar las nuevas soluciones a partir de las parejas que vamos escogiendo del vector. Si tenemos la pareja (i,j) quitaremos un porcentaje del peso a la posición i y se la sumaremos a la posición j (el ratio usado en la práctica es 0.4). Esto nos genera una nueva solución la cual comprobaremos si es válida y, en caso de ser válida, si supera el fitness de nuestra mejor solución hasta el momento. Este nuevo vector solución con las nuevas modificaciones en los índices (i,j) se convertirá en nuestra nueva solución. Dejaremos de recorrer el vector de vecinos y volveremos a generar un vecindario mezclado aleatoriamente para recorrerlo de nuevo desde el principio para ver si a partir de esta nueva solución generada conseguimos una mejor.

Si la solución generada no es válida o no mejora la solución actual, restauraremos las modificaciones hechas al vector solución. De esta manera podemos usar un solo vector y modificar dos posiciones en vez de hacer copias de vectores completos.

Mientras que no se superen las restricciones de las evaluaciones y se sigan encontrando vecinos mejores en cada vecindario seguiremos intentando encontrar la mejor solución. En caso de recorrer todo el vecindario y no encontrar una solución mejor o que se superen las evaluaciones permitidas, se detendrá la búsqueda.

3.2 Algoritmo Genético Generacional

Para el algoritmo genético generacional se ha construido una clase **Agg** para implementar la estructura del algoritmo y dos clases hijas **AggArit** y **AggBLX** las cuales heredan de **Agg** y solo modifican la función de cruce. De esta manera podemos tener una única implementación del algoritmo y cambiar el operador de cruce con facilidad.

```
# Pseudocódigo del algoritmo de búsqueda local
función Agg(max_evals):
    evals = 0
    bestFitness = -infinito
    bestSolution
    poblation = soluciones aleatorias
    for i in range(0, populationSize):
      tempfitness = fitness(poblation[i])
      evals += 1
      fitnesspoblation.pushback(tempfitness)
      if tempfitness > bestFitness:
        bestFitness = tempfitness
        bestSolution = poblation[i]

    while evals < maxevals:
      tournament(tournamentsize, populationsize, populationsize,
fitnesspoblation)
```

```
      expectedparentscrossover = redondeo hacia arriba(crossoverprob *
idxparents.size())
      for i in range(0, expectedparentscrossover, 2):
        son = crossover(poblation[idxparents[i]], poblation[idxparents[i +
1]])
        reparar hijos inválidos
        sons.pushback(son.first)
        sons.pushback(son.second)

    for i in range(expectedparentscrossover, poblationsize):
      sons.pushback(poblation[idxparents[i]])

    ## mutación
    expectedmutation = mutationprob * populationsize
    for i in range(0, expectedmutation):
      idxsolution = random entre 0 y populationsize - 1
      mutsol = sons[idxsolution]
      mutation(mutSol, problem, 0.15, problemSize * 5)
      sons[idxSolution] = mutSol

    # Sustitución de la población
    for i in range(0, populationsize - 1):
      tempfitness = fitness(sons[i])
      evals += 1
      fitnesspoblation.pushback(tempfitness)
      if tempfitness > bestfitnessSons:
        bestfitnessSons = tempfitness
        idxbest = i
      if tempfitness < worstfitnessSons:
        worstFitnessSons = tempfitness
        idxworst = i
      if evals == maxevals:
        terminar
    if bestfitness > worstfitnesssons:
      sons[idxworst] = bestsolution
      fitnesspoblation[idxworst] = bestfitness
    if bestfitnessSons > bestfitness:
      bestsolution = sons[idxbest]
      bestfitness = bestfitnessSons
    poblation = sons
return bestsolution, bestfitness, evals
```

3.3 Algoritmo Genético Estacionario

Para el algoritmo genético estacionario se ha construido una clase **Age** para implementar la estructura del algoritmo y dos clases hijas **AgeArit** y **AgeBLX** las cuales heredan de **Age** y solo modifican la función de cruce. De esta manera podemos tener una única implementación del algoritmo y cambiar el operador de cruce con facilidad.

3.4 Algoritmo Memético

Para la implementación de los algoritmos meméticos he usado una clase llamada **Am** en la cual implemento el método optimize general, la búsqueda local y el operador de cruce. Dentro de esta clase hay un método virtual **applyLocalSearch** que es el que marca la diferencia entre los tres tipos de algoritmo genético. Este método lo implementa cada subclase del algoritmo memético. A continuación describo el método applyLocalSearch que usa cada uno.

3.4.1 AM-Random

```
# Pseudocódigo de AM-Random (applyLocalSearch)
función applyLocalSearch(poblation, fitnesspoblation, bestFitness,
bestsolution, problem):
    nRand = redondeo hacia arriba(poblation.size() * 0.10)
    evals = 0
    for i in range(0, nRand):
      result = localSearch(problem, 100, poblation[i])
      evals += 100
      if result.fitness > fitnesspoblation[i]:
        fitnesspoblation[i] = result.fitness
        poblation[i] = result.solution
      if result.fitness > bestFitness:
        bestFitness = result.fitness
        bestsolution = result.solution
    return evals
```

3.4.2 AM-All

```
# Pseudocódigo de AM-All (applyLocalSearch)
función applyLocalSearch(poblation, fitnesspoblation, bestFitness,
bestsolution, problem):
    evals = 0
    for i in range(0, populationSize):
      result = localSearch(problem, 100, poblation[i])
      if result.fitness > fitnesspoblation[i]:
        fitnesspoblation[i] = result.fitness
        poblation[i] = result.solution
      if result.fitness > bestFitness:
        bestFitness = result.fitness
        bestsolution = result.solution
      evals += 100
    return evals
```

3.4.3 AM-Best

```
# Pseudocódigo de AM-Best (applyLocalSearch)
función applyLocalSearch(poblation, fitnesspoblation, bestFitness,
bestsolution, problem):
```

```
evals = 0
nBest = redondeo hacia arriba(populationSize * 0.10)
 indices = índices de las nBest mejores soluciones según fitnesspoblation
for i in indices:
  result = localSearch(problem, 100, poblation[i])
  evals += 100
  if result.fitness > bestFitness:
    bestFitness = result.fitness
    bestsolution = result.solution
    poblation[i] = result.solution
return evals
```

4 Estructura del código

El código de la práctica está organizado en una estructura modular que facilita la compilación y ejecución. A continuación se describe la organización del proyecto y el manual de usuario para compilar y ejecutar el programa.

4.1 Organización de archivos

El proyecto está organizado de la siguiente manera:

- `software/` : Directorio raíz que contiene todo el código fuente
  - `main.cpp` : Archivo principal que ejecuta los algoritmos
  - `src/` : Contiene las implementaciones (.cpp)
    - `age.cpp` : Implementación del algoritmo AGE
    - `agg.cpp` : Implementación del algoritmo AGG
    - `amall.cpp` : Implementación del algoritmo AM-All
    - `ambest.cpp` : Implementación del algoritmo AM-Best
    - `amrand.cpp` : Implementación del algoritmo AM-Rand
    - `pportfolio.cpp` : Implementación de la clase del problema
    - `randomsearch.cpp` : Implementación del algoritmo de búsqueda aleatoria
    - `greedy.cpp` : Implementación del algoritmo Greedy
    - `localsearch.cpp` : Implementación del algoritmo de búsqueda local
    - `operator.cpp` : Implementación de la clase Operator
  - `include/` : Contiene los archivos de cabecera (.h y .hpp)
    - `age.h`: Clase base para los algoritmos genéticos estacionarios
    - `agg.h`: Clase base para los algoritmos genéticos generacionales
    - `agearit.h`, `ageblx.h`: Clases hijas de `age.h` con los correspondientes operadores
    - `aggarit.h`, `aggblx.h`: Clases hijas de `agg.h` con los correspondientes operadores
    - `amall.h`, `ambest.h`, `amrand.h`: Clases de los algoritmos meméticos.
    - `pportfolio.h`, `greedy.h`, `localsearch.h`, `randomsearch.h`
    - `operator.h` : Clase de operadores genéticos
    - `problem.h` : Clase base abstracta
    - `mh.h` : Clase base para los algoritmos de metaheurísticas
    - `solution.h` : Estructura para representar soluciones
    - `random.hpp`, `utils.hpp` : Utilidades auxiliares

  - `Eigen/` : Biblioteca para operaciones con matrices y vectores
- `data/` : Contiene los archivos CSV con los datos de entrada
- `CMakeLists.txt` : Archivo de configuración de CMake
- `script.sh` : Script de compilación automatizada
- `Makefile` : Archivo generado automáticamente por CMake

4.2 Compilación

El proyecto utiliza **CMake** como sistema de compilación automática. Se proporcionan dos modos de compilación:

Opción 1: Usando el script de compilación

El script **script.sh** automatiza el proceso de compilación:

```
# Para compilar en modo debug (con símbolos de depuración):
./script.sh debug

# Para compilar en modo release (optimización máxima):
./script.sh release
```

- **Modo debug**: Compila con la flag -O0 (sin optimización) y -g (símbolos de depuración), para detectar errores y depurar.
- **Modo release**: Compila con -O3 (máxima optimización) y -DNDEBUG (desactiva comprobaciones de Eigen), con esta compilación se han hecho las pruebas.

Opción 2: Usando CMake directamente

```
# Configurar el proyecto en modo release:
cmake CMakeLists.txt -DCMAKE_BUILD_TYPE=Release

# Compilar:
make
```

Tras cualquiera de estos procesos se generará el ejecutable **program** en el directorio software/.

4.3 Manual de usuario: Ejecución

Una vez compilado el programa, se puede ejecutar de las siguientes formas:

Ejecución estándar (comparación de algoritmos):

```
./program <ruta_archivo_csv> <low> <high> <evaluaciones> <seed> <output_csv>
```

Parámetros:

- **ruta_archivo_csv**: Ruta al archivo CSV con los datos de retornos diarios (ejemplo: data/portfolio.csv)
- **low**: Porcentaje mínimo de inversión permitido por empresa (ejemplo: 0.005)
- **high**: Porcentaje máximo de inversión permitido por empresa (ejemplo: 0.08)
- **evaluaciones**: Número de ejecuciones de cada algoritmo con diferentes seeds

- **seed**: Semilla inicial para el generador de números aleatorios (se incrementa en 10 por cada evaluación)
- **output_csv** : Archivo CSV donde se guardan los fitness de todas las ejecuciones

El parámetro output_csv es opcional, si no se pone entonces no se guardan los datos en un fichero y solo aparecen las medias.

Ejemplo de ejecución:

```
./program data/syp_100.csv 0.005 0.05 50 42 syp100.csv
```

Este comando ejecutará cada uno de los tres algoritmos (Random Search, Greedy y Local Search) 5 veces con seeds diferentes (42, 52, 62, 72, 82). Al finalizar mostrará para cada algoritmo:

- Fitness medio obtenido
- Fitness del año 2025
- Beneficio obtenido en el año 2025
- Número medio de evaluaciones realizadas
- Tiempo medio de ejecución (en segundos)

Al final creará un .csv con los datos de la ejecución

Ejecución simplificada (valores por defecto):

```
./program data/syp_100.csv 0.005 0.05
```

Ejecuta una sola evaluación con seed 42.

Ejecución de ayuda:

```
./program
```

Muestra el mensaje de ayuda con las instrucciones de uso.

5 Experimentos y análisis de los resultados

5.1 Casos empleados y valores de los parámetros.

Para comparar los resultados de los algoritmos se han usado los beneficios diarios de las empresas del S&P500, S&P100 e IBEX35 con los siguientes parámetros.

- **λ**= 500 (usado en la función fitness)
- **seed** = 42 (por cada ejecución del algoritmo se incrementa 10 unidades)
- **low** = 0.005 (para todos los datos)
- **high**(para IB35) = 0.08
- **high**(para S&P100) = 0.05
- **high**(para S&P500) = 0.02
- **totalevaluations** = 50

La ejecución se ha hecho como se indicó en el apartado anterior, cambiando en los argumentos del programa el archivo de entrada de datos y ajustando su respectivo valor **high** correspondiente a cada archivo.

5.2 Resultados obtenidos

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	−4.649	−3.386	0.115	1	0.001
Random	−4.980	−3.289	0.096	10000	0.005
BL	−4.608	−3.109	0.019	1517.36	0.001
AGG-Arit	−4.657	−3.186	0.045	10000	0.002
AGG-BLX	−4.500	−3.123	0.046	10000	0.003
AGE-Arit	−4.576	−3.158	0.041	10000	0.003
AGE-BLX	−4.495	−3.096	0.040	10000	0.003
AM-ALL	−4.496	−3.089	0.030	10000	0.026
AM-RAND	−4.498	−3.090	0.030	10000	0.006
AM-BEST	−4.497	−3.084	0.032	10000	0.005

Tabla 1: Resultados de Ibex 35 (Medias)

Algoritmo	**2015-2024 Fitness**	**2025 Fitness**	**Beneficio**	**Evaluaciones**	**Tiempo (segundos)**
Greedy	−3.663	−5.020	0.009	1	0.001
Random	−3.408	−3.968	0.069	10000	0.02
BL	−3.118	−3.918	0.094	8984.76	0.01
AGG-Arit	−3.184	−3.918	0.094	10000	0.012
AGG-BLX	−2.866	−3.709	0.094	10000	0.014
AGE-Arit	−3.127	−3.869	0.108	10000	0.014
AGE-BLX	−2.870	−3.770	0.108	10000	0.012
AM-ALL	−2.8744	−3.722	0.103	10000	0.289
AM-RAND	−2.945	−3.786	0.084	10000	0.042
AM-BEST	−2.945	−3.697	0.113	10000	0.044

Tabla 2: Resultados de S&P100 (Medias)

Algoritmo	2015-2024 Fitness	2025 Fitness	Beneficio	Evaluaciones	Tiempo (segundos)
Greedy	−3.438	−5.204	−0.062	1	0.001
Random	−3.638	−4.606	−0.0583	10000	0.253
BL	−3.517	−4.313	0.018	10000	0.220
AGG-Arit	−3.321	−4.170	0.003	10000	0.189
AGG-BLX	−3.003	−4.097	−0.009	10000	0.191
AGE-Arit	−3.338	−4.431	0.005	10000	0.202
AGE-BLX	−3.095	−4.148	−0.028	10000	0.184
AM-ALL	−3.230	−4.186	−0.011	10000	2.687
AM-RAND	−3.2199	−4.159	−0.017	10000	0.446
AM-BEST	−3.227	−4.123	−0.017	10000	0.456

Tabla 3: Resultados de S&P500 (Medias)

Los valores presentados en la tabla corresponden a las medias de 5 ejecuciones independientes con diferentes semillas aleatorias.

5.3 Boxplots de los resultados

A continuación se muestran los boxplots de los resultados obtenidos para cada conjunto de datos tras 50 ejecuciones de cada algoritmo:

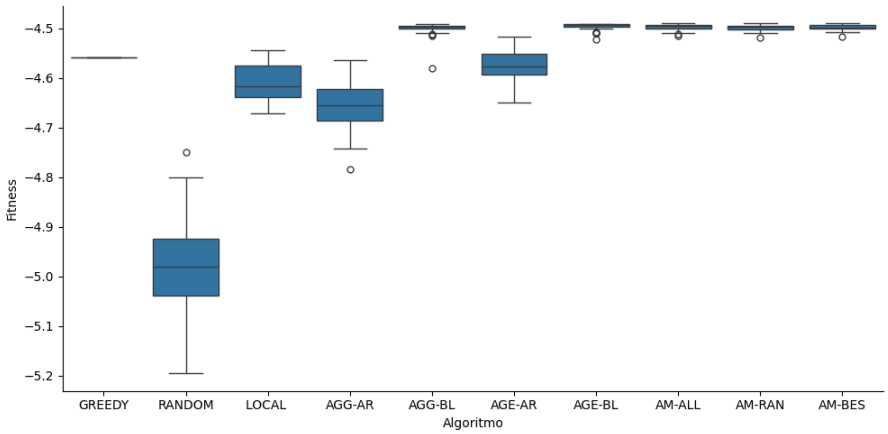


Figura 1: Boxplot de resultados para Ibex 35

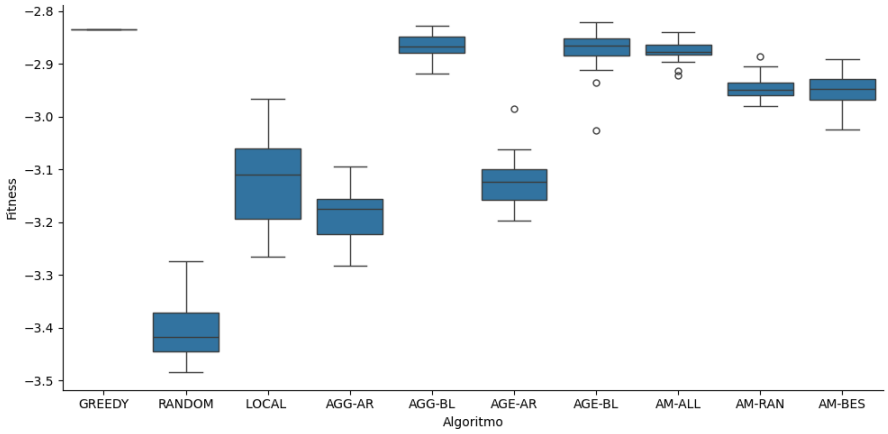


Figura 2: Boxplot de resultados para S&P100

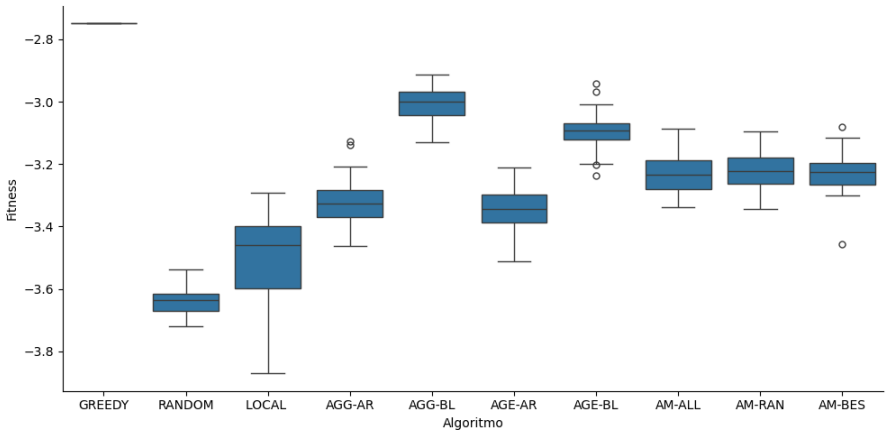


Figura 3: Boxplot de resultados para S&P500

6 Análisis de los resultados

Al observar los resultados obtenidos podemos ver que cada algoritmo se comporta de manera muy diferente dependiendo del conjunto de datos usado. Para poder entender por qué unos algoritmos funcionan mejor que otros es necesario analizar cómo trabaja cada uno y qué características del problema aprovechan o ignoran.

6.1 Comportamiento de los Algoritmos Genéticos Generacionales

Los AGG reemplazan en todas las iteraciones la población al completo, conservando el mejor de la población anterior e intercambiándolo por el peor de la generación que entra. Este tipo de reemplazo hace que haya más diversidad de soluciones y por tanto el algoritmo tenga menos riesgo de quedarse «atascado» en una zona del espacio de soluciones. Esto hace que el algoritmo explore más parte del espacio de soluciones, en consecuencia se pierden soluciones que, aunque no fuesen la mejor, eran buenas y podrían haber convergido a mejores soluciones mediante cruces y mutaciones.

Podemos ver en el

6.2 Comportamiento de los Algoritmos Genéticos Estacionarios

6.3 Comportamiento de los Algoritmos Meméticos

6.4 Mejor operador para los genéticos

Como se puede observar en los resultados, en ambos algoritmos genéticos el mejor resultado lo obtiene el operador **BLX-α**. La principal diferencia entre este y el aritmético es que el **BLX-α** permite que el gen resultante de los hijos al cruzar los genes de los padres pueda estar fuera del intervalo que genera el mínimo y el máximo de los padres, por tanto los hijos resultantes de los cruces se pueden alejar más de sus padres y permiten explorar más el espacio de soluciones. Por otra parte, el cromosoma resultante del cruce aritmético contiene genes que no pueden tener un valor mayor o menor que el mayor o menor de los padres (al ser una combinación lineal) por lo que al aplicar este tipo de cruce, los hijos convergen a la media de los padres obteniendo mejor resultado en entornos explotativos.

6.5 Búsqueda Local vs Algoritmos Genéticos

Los algoritmos genéticos en el problema del portfolio obtienen mejores resultados en el problema del portfolio ya que exploran más el espacio de soluciones que la búsqueda local. La búsqueda local es más explotativa por lo que le es fácil entrar en máximos locales en datasets pequeños como el Ibex 35 mientras que los algoritmos genéticos exploran un mayor rango de soluciones por lo que tienen más facilidad de evitar máximos locales. También cabe destacar que la búsqueda local se ve muy afectada por la solución inicial que se toma, al ser esta aleatoria, dependemos más de la «suerte» a la hora de encontrar una mejor solución antes de las evaluaciones permitidas. En los algoritmos genéticos también influye la población inicial, pero sobre todo en los AGG, al reemplazar toda la población en cada iteración y conservar la mejor solución, se consigue una mayor diversidad y mayor probabilidad de no quedarse atascado en un máximo local.

6.6 AGE-BLα vs AGG-BLα

En cuanto a comparación entre algoritmos genéticos nos fijaremos en los que usan el operador **BLX-α** para compararlos en igualdad de condiciones cuando usan el mejor operador.

Se puede observar facilmente en los gráficos de lineas que ambos algoritmos tienen resultados muy parecidos y de los mejores en comparación a los otros algoritmos, en datasets pequeños como podría ser el del ibex35 vemos que no se aprecia apenas diferencia ya que la ventaja explorativa del AGG no tiene apenas efecto. La diferencia se puede observar en el dataset de mayor tamaño donde vemos que el AGG (linea morada) consigue mantener su fitness por encima del AGE en la mayoría de las ejecuciones. Por tanto podemos ver que en el SP500 se obtienen mejores soluciones cuando se tiene mayor

diversidad en la población del algoritmo que cuando se intenta conservar de manera prematura a los mejores indivíduos (AGE).

6.7 Comparación entre meméticos

6.8 Meméticos vs mejor AG

6.9 Comparación gráfica fitness 2014-2024 (50 ejecuciones)

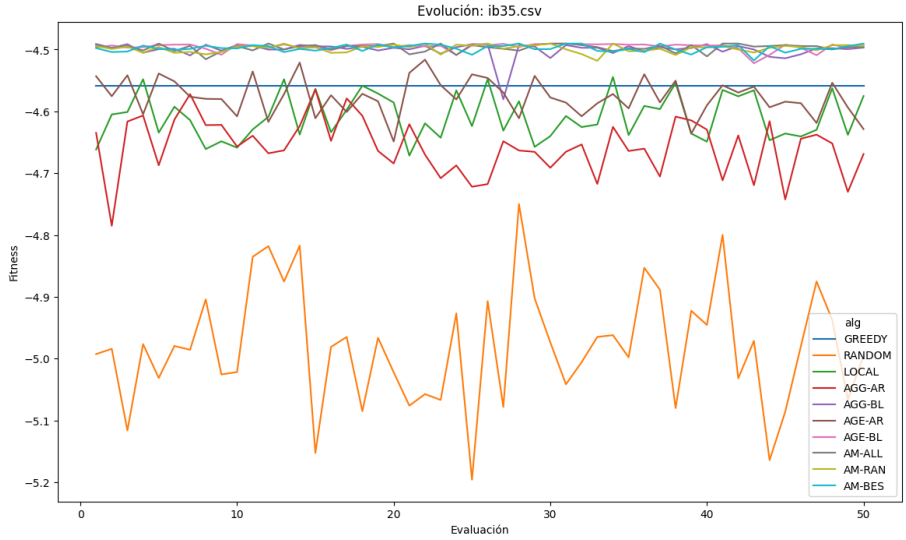


Figura 4: Comparación fitness para Ibex 35

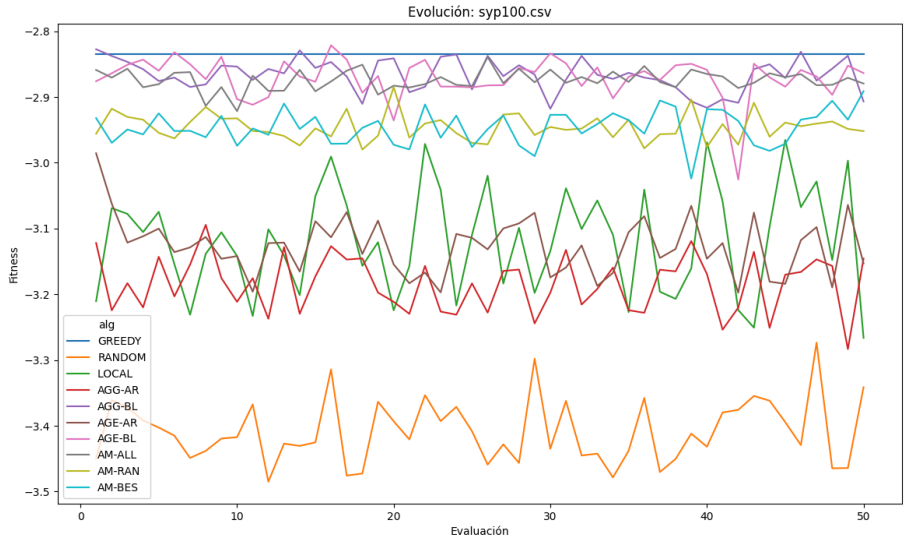


Figura 5: Comparación fitness para S&P100

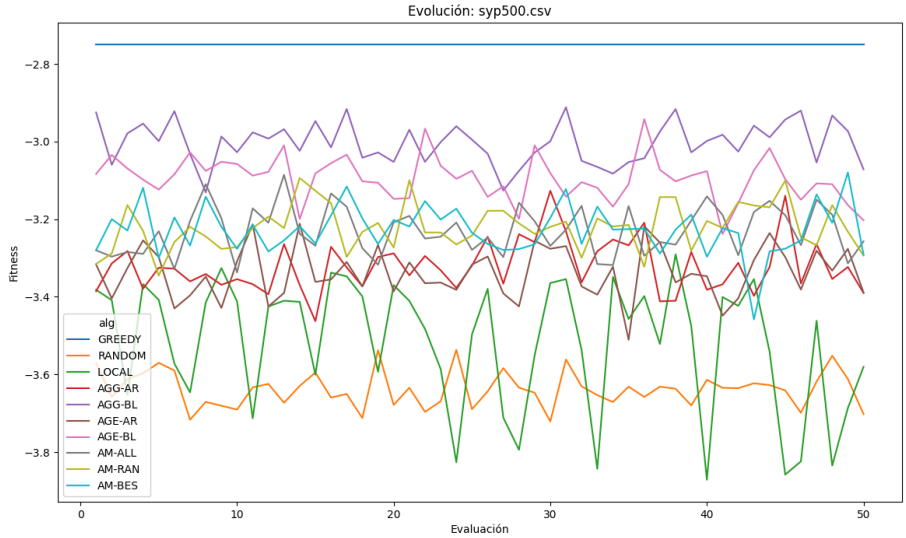


Figura 6: Boxplot de resultados para S&P500

En estas gráficas podemos ver la evolución del fitness a través de las 50 ejecuciones hechas, vemos que como es lógico en el Algoritmo Greedy se mantiene constante y podemos ver cómo la variabilidad del fitness va disminuyendo en el algoritmo de Búsqueda Aleatoria conforme el conjunto de datos aumenta, sin embargo la variabilidad del fitness de la búsqueda local sigue una distribución parecida en todos los casos.

Podemos observar también que en algunas ejecuciones la Búsqueda aleatoria ha obtenido mejor fitness que la Búsqueda Local, esto ocurre mayoritariamente en los datos del S&P500 ya que al ser un espacio de soluciones tan grande a la Búsqueda Local se le acaban las evaluaciones muy rápido y en algunos casos puede sacar fitness peores que la Búsqueda Aleatoria.

6.10 Comparación global y conclusiones